

Từ NumPy tự viết tới PyTorch thành thạo (From Hand-Written NumPy to Fluent PyTorch)

PHỤ THUỘC A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (quy tắc chuỗi); B/09-thiet-ke-ham-mat-mat (*hinge loss*, cross-entropy); B/11-gioi-han-tinh-toan (*roofline* — bắt buộc trước khi đọc phần tăng tốc của mục 19.2); C/12-co-che-huan-luyen (BatchNorm, dropout, optimizer).

TÓM TẮT

Chương này có hai giai đoạn đọc theo đúng thứ tự: tự cài **lan truyền ngược** (*backpropagation*) bằng NumPy để có trực giác về cơ chế, rồi học PyTorch theo mạch vấn đề → công cụ để có năng suất. Đọc xong bạn debug được những lỗi không có thông báo lỗi: gradient sai, trạng thái train/eval lẫn lộn, rò bộ nhớ vì logging. Bạn cũng tăng tốc được một vòng lặp huấn luyện dựa trên số đo, thay vì dựa trên mẹo nghe được. Nếu chỉ nhớ một điều: giai đoạn 1 là bài tập *trực giác* tăng [T2][T3], làm đúng một lần rồi bỏ. Giai đoạn 2 là bài tập *công cụ* tăng [T4], phải cập nhật liên tục. Lẫn lộn hai loại này là cách phổ biến nhất để phí thời gian.

KIẾN THỨC CẦN CÓ TRƯỚC

- **lượt xuôi** (*forward pass*) và **lượt ngược** (*backward pass*) — forward là lượt tính từ đầu vào ra loss; lượt ngược là lượt tính ngược để lấy đạo hàm của loss theo từng tham số. Mọi khung huấn luyện đều là vòng lặp hai lượt này.
- **gradient** — vector đạo hàm của loss theo tham số; nó chỉ hướng làm loss *tăng* nhanh nhất, nên ta đi ngược lại nó.
- **lan truyền ngược** (*backpropagation*) — thuật toán tính toàn bộ gradient bằng một lượt đi ngược qua đồ thị tính toán, tốn cùng bậc chi phí với một lượt xuôi — lý do duy nhất khiến huấn luyện mạng sâu khả thi.
- **learning rate** — độ dài mỗi bước cập nhật trọng số theo hướng ngược gradient. Quá lớn thì loss phát tán, quá nhỏ thì huấn luyện không bao giờ xong; đây là siêu tham số quan trọng nhất.
- **framework** — thư viện lo hộ bạn đồ thị tính toán, lan truyền ngược tự động, quản lý thiết bị và dữ liệu — PyTorch là ví dụ. Giá trị của chương này nằm ở chỗ biết framework đang giấu cái gì.
- **tăng bán rã [T2] so với [T4]** — [T2] là kiến thức nguyên lý, không hết hạn khi công cụ đổi; [T4] là kiến thức công cụ, sẽ cũ trong vài năm. Giai đoạn 1 của chương là [T2]–[T3], giai đoạn 2 là [T4].

CÂU HỎI MỞ ĐẦU

- Q1. Một mạng 10^9 tham số huấn luyện được trong vài ngày, trong khi vi phân thuận cần 10^9 lượt chạy — lượt ngược lấy khoản tiết kiệm chín bậc đó ở đâu ra?
- Q2. Số trọng số y hệt nhau, vậy vì sao huấn luyện tốn bộ nhớ gấp nhiều lần suy luận?
- Q3. Kiểm tra gradient của bạn qua sạch mọi tầng nhưng mạng vẫn không học — phép kiểm đó theo thiết kế đã bỏ sót cái gì?
- Q4. Bạn bật torch.compile và AMP, mọi thứ chạy đúng, nhưng thời gian một *epoch* không đổi một giây nào. Vì sao đó không phải là bug?

Framework che giấu rất nhiều thứ, và phần lớn thời gian đó là điều tốt. Nhưng có một tập nhỏ những thứ mà không tự tay làm một lần thì bạn sẽ không bao giờ debug nổi. Chúng không hiện ra dưới dạng thông báo lỗi, chỉ dưới dạng một con số hơi lệch hoặc một đường cong loss hơi chậm. Câu hỏi dẫn dắt của chương là: *tập nhỏ ấy gồm những gì, và làm tới đâu thì đủ?*

Chương chia làm hai mục vì hai giai đoạn có *bản chất khác nhau*, và nhằm lẫn giữa chúng là cái bẫy chính:

- **19.1 — Giai đoạn 1, tự viết bằng NumPy** [T2][T3]. Mục tiêu là trực giác về cơ chế, không phải kỹ năng công cụ. Bạn sẽ không dùng lại một dòng NumPy nào trong việc thật. Có tiêu chí dừng rõ ràng, và khi đạt thì *dừng hẳn*.
- **19.2 — Giai đoạn 2, PyTorch** [T4]. Mục tiêu là năng suất và khả năng chẩn đoán. Nội dung sẽ cũ khi API đổi, nhưng danh sách các nút thắt mà công cụ gỡ thì không cũ.

Thứ tự phụ thuộc chỉ đi một chiều. Mục 19.2 giới thiệu autograd, buffer và cặp `train()/eval()` như *tên gọi mới* của những thứ bạn vừa tự cài bằng tay. Đọc ngược thì chúng thành từ vựng phải học thuộc, thay vì thành khái niệm đã hiểu. Ngược lại, nếu bạn đã từng viết một solver phi tuyến với Jacobian giải tích và bước so sai phân, giai đoạn 1 sẽ nhanh hơn bạn nghĩ. Nó là cùng một kỹ năng, đặt vào miền mới.

Một sợi chỉ chạy suốt cả chương: hầu hết bug ở đây là bug về *trạng thái*, không phải về công thức. Ba câu hỏi lặp lại ở cả hai giai đoạn. Cái gì được nhớ giữa lượt xuôi và lượt ngược? Cái gì được lưu vào checkpoint? Cái gì đổi hành vi giữa `train` và `eval`? Ở giai đoạn 1 chúng hiện ra dưới dạng “tôi phải cache gì”; ở giai đoạn 2 chúng mang tên `parameter` so với `buffer`.

Bản đồ chương 19 — Từ NumPy tự viết tới PyTorch thành thạo

Chương này chia làm hai mục vì hai giai đoạn có *bản chất khác nhau* và có tuổi thọ khác nhau. Nhằm lẫn giữa chúng là cái bẫy chính của cả chương.

19.1 — Giai đoạn 1: tự viết bằng NumPy [T2][T3] (*cốt lõi; làm đúng một lần rồi bỏ*). Cài linear classifier với cả hinge lẫn softmax, viết lượt ngược giải tích cho sáu tầng, kiểm tra gradient số cho từng tầng trước khi ghép, rồi ghép thành mạng n tầng với SGD + momentum. Đây *không* phải bài tập công cụ — mục tiêu là trực giác về cơ chế, thứ không hết hạn khi framework đổi. Tiêu chí dừng rất rõ: giải thích được vì sao BatchNorm khiến lượt ngược tốn thêm bộ nhớ, và vì sao dropout phải scale lúc train. Đạt rồi thì chuyển sang 19.2 và không viết NumPy nữa.

19.2 — Giai đoạn 2: PyTorch thành thạo [T4] (*cốt lõi phần ánh xạ triệu chứng → công cụ; tham khảo phần cú pháp*). Chín nút thắt và công cụ gỡ từng cái: `stride` và `view/copy`, autograd, `parameter` so với `buffer`, `DataLoader`, tự viết vòng lặp một lần, AMP và DDP và `torch.compile`, profiler, tính tắt định, và cuối cùng là kernel tùy biến (biết là đủ). Phần bèn của mục này là cây quyết định tăng tốc; phần sẽ cũ là tên API.

Vì sao chia như vậy và thứ tự bắt buộc. 19.2 giới thiệu autograd, buffer, `train()/eval()` như tên gọi mới của thứ bạn vừa tự cài trong 19.1. Đọc ngược sẽ biến chúng thành từ vựng phải học thuộc thay vì khái niệm đã hiểu.

Sợi chỉ xuyên chương. Hầu hết bug ở đây là bug về *trạng thái*, không phải công thức: cái gì được nhớ giữa lượt xuôi và lượt ngược, cái gì được lưu vào checkpoint, cái gì đổi hành vi giữa hai chế độ.

Phụ thuộc lên trên. A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (quy tắc chuỗi), B/09-thiet-ke-ham-mat-mat, B/11-gioi-han-tinh-toan (bắt buộc trước phần tăng tốc), C/12-co-che-huan-luyen.

Toả xuống dưới. D/20-toi-uu-va-nen-mo-hinh dùng lại đúng những đòn bẩy này nhưng cho **suy luận** (*inference*); D/21-inference-va-trien-khai dùng lại kỹ năng phân biệt “lệch do bug” với “lệch do số học”; D/22-tai-lap-va-mlops mở rộng mục về tính tất định thành một chương.

I 1.1 Giai đoạn 1: tự viết bằng NumPy (Building It Yourself in NumPy)

PHỤ THUỘC A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (quy tắc chuỗi, Jacobian); B/09-thiet-ke-ham-mat-mat (*hinge loss* và *cross-entropy*); C/12-co-che-huan-luyen/01-co-hoc-cua-gradient và 02-lam-cho-training-chay-duoc (BatchNorm, dropout — ở đây ta cài lại chúng bằng tay).

TÓM TẮT

Mục này là bài tập một lần: tự cài **lan truyền ngược** (*backpropagation*) bằng NumPy cho sáu tầng, kiểm tra từng tầng bằng **kiểm tra gradient số** (*numerical gradient check*), rồi ghép thành một mạng chạy được. Nó không phải bài tập công cụ mà là bài tập trực giác tầng [T2][T3]: đọc xong bạn giải thích được vì sao bộ nhớ huấn luyện lớn hơn bộ nhớ **suy luận** (*inference*) nhiều lần, vì sao quên `eval()` lại đổi kết quả, và bạn tự chẩn đoán được một gradient sai thay vì đoán mò. Nếu chỉ nhớ một điều: không ghép một tầng chưa qua kiểm tra gradient — vì kiểm từng tầng giữ không gian nghi vấn ở đúng một phần tử, còn ghép trước kiểm sau làm nó phình lên thành số tầng nhân số tương tác.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm mất mát (loss function)** — hàm biến đầu ra của mô hình và đáp án đúng thành một số thực đo mức sai. Huấn luyện là cực tiểu hoá số đó.
- **gradient và quy tắc chuỗi** — gradient là vector đạo hàm riêng của loss theo từng tham số. Vì mạng là hàm hợp của nhiều tầng, đạo hàm được tính bằng quy tắc chuỗi — nhân dồn các đạo hàm cục bộ từ đầu ra ngược về đầu vào.
- **kích hoạt (activation)** — đầu ra trung gian của một tầng. Lượng bộ nhớ huấn luyện phần lớn là chỗ chứa các kích hoạt này, vì lượt ngược cần đọc lại chúng.
- **ReLU, dropout, BatchNorm** — ReLU là hàm phi tuyến $\max(0, x)$; dropout tắt ngẫu nhiên một phần kích hoạt lúc huấn luyện để chống *overfitting*; BatchNorm chuẩn hoá kích hoạt theo thống kê của *batch* để việc huấn luyện ổn định hơn. Cả ba đều có hành vi *khác nhau* giữa lúc train và lúc test.
- **batch** — nhóm mẫu được xử lý cùng lúc trong một bước cập nhật. Kích thước batch ảnh hưởng tới bộ nhớ, tốc độ, và — với BatchNorm — tới cả kết quả.
- **SGD và momentum** — *stochastic gradient descent* cập nhật trọng số theo gradient của một batch; *momentum* cộng dồn hướng đi của các bước trước để giảm dao động và vượt qua vùng phẳng.
- **float32 so với float64** — hai độ chính xác dấu phẩy động. float32 có epsilon máy $\approx 1,2 \times 10^{-7}$, float64 khoảng $2,2 \times 10^{-16}$; khác biệt này quyết định việc kiểm tra gradient số có nói thật hay không.

A. Đặt vấn đề

Framework che giấu rất nhiều thứ, và phần lớn thời gian đó là điều tốt — không ai muốn viết lại `conv2d` mỗi sáng. Nhưng có một tập nhỏ những thứ mà nếu không tự tay làm một lần thì bạn sẽ không bao giờ debug nổi, vì triệu chứng của chúng không bao giờ xuất hiện dưới dạng một thông báo lỗi. Mạng vẫn chạy, loss vẫn giảm, chỉ là giảm chậm hơn đáng ra, hoặc kết quả lúc `eval()` lệch khỏi lúc train một cách không giải thích được. Không có exception nào để bắt, không có *stack trace* nào để đọc.

Cần nói rõ ngay từ đầu về *loại* bài tập này, vì hiểu sai loại là cách phổ biến nhất để lãng phí một tháng:

- **Đây không phải bài tập tăng bốn** — không phải bài tập công cụ. Bạn sẽ không dùng lại một dòng NumPy nào trong công việc thật, và đó là chuyện bình thường.
- **Đây là bài tập tăng hai và ba** [T2][T3]: mục tiêu là *trực giác về cơ chế*, thứ không hết hạn khi framework đổi phiên bản, khi PyTorch bị thay bằng cái khác, hay khi kiến trúc thời thượng đổi tên.
- **Làm một lần, đạt tiêu chí dừng, rồi không quay lại.** Cái bấy quen thuộc của người học theo CS231n là yêu quá trình này và tiếp tục viết NumPy sau khi nó đã hết tác dụng — lúc đó nó thành một sở thích thú công chứ không còn là học.

Học xong thì làm được gì mà trước đó không làm được? Bạn nhìn một đường cong loss bất thường và khoanh vùng được ngay nó là lỗi gradient, lỗi trạng thái hay lỗi dữ liệu; bạn đọc thông báo hết bộ nhớ GPU và biết tầng nào đang giữ activation; và bạn viết được một `autograd.Function` tùy biến trong PyTorch mà không phải sao chép mẫu từ diễn đàn.

Người đọc đã viết C++ và CUDA có một lợi thế ở đây, và cũng có một cái bấy riêng. Lợi thế là bạn đã quen với việc một hệ thống số học có thể sai ở mức không nhìn thấy được. Cái bấy là bạn dễ bỏ qua bước kiểm tra gradient vì thấy nó buồn tẻ. Đúng như một người viết solver phi tuyến để bỏ bước so Jacobian giải tích với Jacobian sai phân, rồi mất ba ngày cho một dấu trừ.

TRỰC GIÁC

Bạn đã làm chính xác bài tập này rồi, chỉ là ở lĩnh vực khác. Khi thêm một loại *residual* mới vào một bài toán bình phương tối thiểu phi tuyến, bạn viết Jacobian giải tích để solver chạy nhanh, rồi *trước khi tin nó*, bạn so với Jacobian tính bằng sai phân hữu hạn trên vài điểm ngẫu nhiên. Nếu sai số tương đối nhỏ thì dùng; nếu không thì bạn biết chắc lỗi nằm trong đúng khối vừa viết, chứ không phải ở solver.

Lan truyền ngược là cùng một việc, chỉ khác quy mô: một mạng n tầng là một hàm hợp, **lượt ngược** (*backward pass*) là chế độ nghịch của quy tắc chuỗi — đúng cái mà cộng đồng tối ưu gọi là *adjoint method* — và kiểm tra gradient là bước so Jacobian. Giai đoạn 1 vì thế không dạy bạn khái niệm mới; nó chuyển một kỹ năng bạn đã có sang miền mới, ở quy mô nhỏ tới mức mọi con số còn nhìn được bằng mắt.

B. Bước 1 — linear classifier với hai hàm mất mát

Bắt đầu ở chỗ đơn giản nhất mà vẫn còn thú vị: một phân loại tuyến tính $s = Wx + b$, cài cả *hinge loss* kiểu SVM lẫn *softmax loss* (*softmax cross-entropy loss*), viết cả **lượt xuôi** (*forward pass*) lẫn lượt ngược cho từng cái. Điều bạn sẽ thấy tận mắt — chứ không phải đọc thấy — là hai hàm này có *hành vi gradient khác nhau về bản chất*, không chỉ khác về công thức.

Ví dụ nhỏ nhất. Một mẫu, ba lớp, điểm số $s = (2,0; 0,5; -1,0)$, nhãn đúng là lớp 0.

- **Hinge loss**, lẽ $\Delta = 1$:

$$L = \sum_{j \neq y} \max(0, s_j - s_y + \Delta) = \max(0, -0,5) + \max(0, -2,0) = 0.$$

Loss bằng không, và *gradient cũng bằng đúng không* — mọi phần tử của $\partial L / \partial s$ là 0. Mẫu này đã thoả lẽ; nó không còn nói gì với mô hình nữa.

- **Softmax loss**, cùng bộ điểm số: $e^{2,0} = 7,389$, $e^{0,5} = 1,649$, $e^{-1,0} = 0,368$, tổng 9,406, nên $p = (0,786; 0,175; 0,039)$ và $L = -\ln 0,786 = 0,241$. Gradient có dạng đẹp đến mức đáng nhớ,

$$\partial L / \partial s = p - e_y.$$

$$\partial L / \partial s = (-0,214; +0,175; +0,039).$$

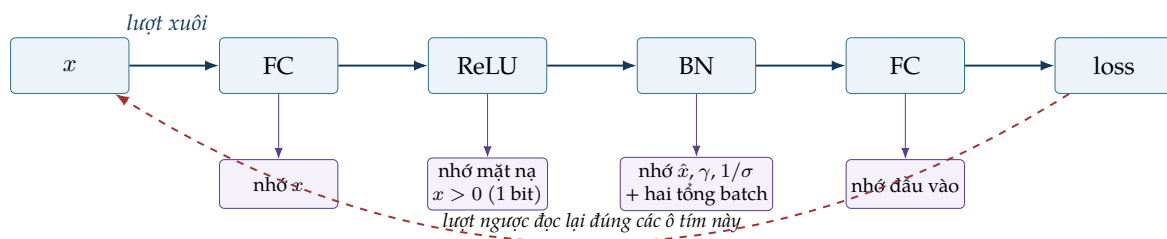
Ý nghĩa của công thức $p - e_y$: gradient chính là “sai lệch giữa niềm tin hiện tại và sự thật”. Không phần tử nào bằng không, nên softmax *luôn* còn đây — nó không có khái niệm “đủ tốt rồi”.

Khác biệt này giải thích rất nhiều hành vi thực nghiệm. Hinge tạo gradient thưa nên ổn định và ít bị chi phối bởi vài mẫu dễ, nhưng nó dừng học ngay khi lẽ được thoả, kể cả khi bạn còn muốn mô hình tự tin hơn. Softmax luôn có gradient nên tiếp tục tinh chỉnh, nhưng chính vì thế nó nhạy với nhãn sai — một mẫu bị gán nhầm sẽ bơm gradient vào suốt quá trình huấn luyện, không bao giờ tắt. Ai đã đọc B/09-thiet-ke-ham-mat-mat sẽ nhận ra đây là cùng trục lập luận đã dẫn tới **focal loss**.

↔ cùng nguyên lý với B/09 — cùng trục lập luận đã dẫn tới focal loss — dời trọng số gradient khỏi những mẫu không còn dạy được gì.

C. Bước 2 — lượt ngược giải tích cho sáu tầng

Bước tiếp theo là viết lượt ngược giải tích cho **fully-connected**, ReLU, dropout, BatchNorm, **convolution** và **max-pooling**. Giá trị của bước này không nằm ở các công thức — chúng có sẵn ở mọi nơi — mà ở chỗ nó buộc bạn trả lời cho từng tầng một câu hỏi mà framework không bao giờ hỏi bạn: *backward của tầng này cần nhớ gì từ lượt xuôi?* Câu trả lời là toàn bộ lý do bộ nhớ huấn luyện lớn hơn bộ nhớ suy luận nhiều lần. Hình 1.1 vẽ đúng cơ chế đó.



Hình 1.1. Vì sao huấn luyện tốn bộ nhớ hơn suy luận: mỗi tầng phải *ghi lại* một phần trạng thái lúc lượt xuôi để lượt ngược đọc lại. Ô tím là thứ không giải phóng được cho tới khi lượt ngược chạy qua. ReLU rẻ (một bit mỗi phần tử), BatchNorm đắt nhất vì ngoài $\hat{x}$ nó còn ràng buộc cả **batch**.

Bảng 1.1. Sáu tầng, và thứ mà lượt ngược của mỗi tầng buộc phải giữ lại từ lượt xuôi.

Tầng	Backward cần nhớ	Chi phí	Chỗ hay sai
Fully-connected	Đầu vào x (cho $\partial L / \partial W$)	Bằng kích thước đầu vào	Quên chuyển vị; quên cộng dồn $\partial L / \partial b$ theo batch
ReLU	Chỉ mặt nạ $x > 0$ (1 bit mỗi phần tử)	Rất rẻ	Xử lý $x = 0$ tùy tiện — chính là điểm gây làm kiểm tra gradient báo động giả
Dropout	Mặt nạ đã dùng lúc lượt xuôi	Bằng kích thước đầu ra	Không dùng inverted dropout , hoặc sinh mặt nạ mới ở lượt ngược
BatchNorm	$\hat{x}, \gamma, 1/\sigma$ — và lượt ngược khớp cả <i>batch</i>	Đắt nhất trong sáu tầng	Coi lượt ngược là phép theo từng phần tử
Convolution	Ma trận im2col hoặc đầu vào để dựng lại nó	Rất đắt nếu vật chất hoá im2col	col2im phải <i>cộng dồn</i> vùng chồng lấn, không phải ghi đè
Max-pool	Chỉ số argmax của mỗi cửa sổ	Rẻ	Xử lý trường hợp bằng nhau khác giữa hai cài đặt

Ba dòng trong Bảng 1.1 đáng dừng lại.

BatchNorm – lượt ngược không phải phép theo từng phần tử

$$\frac{\partial L}{\partial x_i} = \frac{\gamma}{N\sigma} \left[N \frac{\partial L}{\partial y_i} - \sum_j \frac{\partial L}{\partial y_j} - \hat{x}_i \sum_j \frac{\partial L}{\partial y_j} \hat{x}_j \right].$$

Công thức này nói điều gì? Rằng gradient của mẫu i bị trừ đi hai đại lượng tính trên *toàn batch* — một cho trung bình, một cho phương sai. Nói cách khác: BatchNorm không cho phép một mẫu tự do đẩy gradient của nó; mọi mẫu trong batch cùng thoả hiệp. Ba hệ quả bạn sẽ gặp lại suốt sự nghiệp:

- **Bộ nhớ lượt ngược tăng** vì phải giữ cả $\hat{x}$ lẫn các thống kê, và không giải phóng được theo từng mẫu.
- **Batch nhỏ làm gradient nhiễu**, vì ước lượng σ nhiễu — lý do GroupNorm và LayerNorm tồn tại [1], [2].
- **Mô hình huấn luyện với BatchNorm là hàm của cả batch, không phải của từng mẫu** — lý do sâu xa khiến `train()` và `eval()` cho kết quả khác nhau.

Dropout – vì sao phải scale lúc train

Cài theo kiểu *inverted dropout*: lúc train, giữ lại phần tử với xác suất $1 - p$ rồi chia cho $1 - p$; lúc test không làm gì cả. Lý do là bảo toàn kỳ vọng — nếu $\tilde{y} = m \odot y / (1 - p)$ với $m \sim \text{Bernoulli}(1 - p)$ thì $\mathbb{E}[\tilde{y}] = y$, nên đường suy luận của các tầng sau không bị dịch khi ta bật tắt dropout. Cách còn lại (nhân $1 - p$ lúc test) tương đương về toán học nhưng dở về kỹ thuật, vì nó đặt một bước bắt buộc vào đường suy luận — và mọi đường inference đều có ngày bị viết lại, bị *export*, bị port sang C++, và bước ấy sẽ rơi mất. Một điểm trung thực cần ghi: *inverted dropout* chỉ khớp *mô-men bậc một*; phương sai lúc train vẫn lớn hơn lúc test, và đó là một phần lý do dropout tương tác xấu với BatchNorm [3].

Convolution – chỗ nào sinh ra bất định trên GPU

Cách cài dễ hiểu nhất là *im2col*: trải các cửa sổ thành cột rồi gọi một phép nhân ma trận, nên $\partial L / \partial W = \partial L / \partial Y \cdot X_{\text{col}}^\top$ và $\partial L / \partial X_{\text{col}} = W^\top \cdot \partial L / \partial Y$. Bước cuối — *col2im* — phải cộng dồn gradient vào những vị trí mà các cửa sổ chồng lên nhau. Ghi nhớ chi tiết này. Phép cộng dồn tán xạ đó chính là chỗ các cài đặt GPU dùng `atomicAdd`. Cộng số thực dấu phẩy động không có tính kết hợp, nên thứ tự luồng hoàn thành quyết định bit cuối. Đây là một trong những nguồn bất định chính của huấn luyện trên GPU, chủ đề của D/22-tai-lap-va-mlops. Nó cùng một họ nguyên nhân với các lỗi bất định do thứ tự con trỏ mà bạn đã gặp trong SLAM.

↪ **tiền đề** D/22 — cộng số thực không có tính kết hợp, nên mọi phép cộng dồn tán xạ trên GPU là một nguồn bất định — cùng họ nguyên nhân với thứ tự con trỏ trong một hệ thống C++ đa luồng.

D. Bước 3 – kiểm tra gradient, và kỷ luật đi kèm

Mọi công thức ở trên đều có thể sai, và sai một cách im lặng. Cách duy nhất phát hiện là so với sai phân trung tâm:

$$\frac{\partial f}{\partial x_k} \approx \frac{f(x + he_k) - f(x - he_k)}{2h},$$

Ta nhiễu *một* toạ độ về hai phía và xem loss thay đổi bao nhiêu. Rồi tính sai số tương đối $|g_{\text{số}} - g_{\text{giải tích}}| / \max(|g_{\text{số}}|, |g_{\text{giải tích}}|)$.

```
def grad_check(f, x, analytic, h=1e-5, n=20):
```

```
# f: hàm tra về scalar loss; analytic: gradient đã tính giai tích
idx = np.random.choice(x.size, size=n, replace=False)
for k in idx:
    old = x.flat[k]
    x.flat[k] = old + h; fp = f(x)
    x.flat[k] = old - h; fm = f(x)
    x.flat[k] = old
    num = (fp - fm) / (2 * h)
    ana = analytic.flat[k]
    rel = abs(num - ana) / max(1e-12, abs(num), abs(ana))
print(k, num, ana, rel)
```

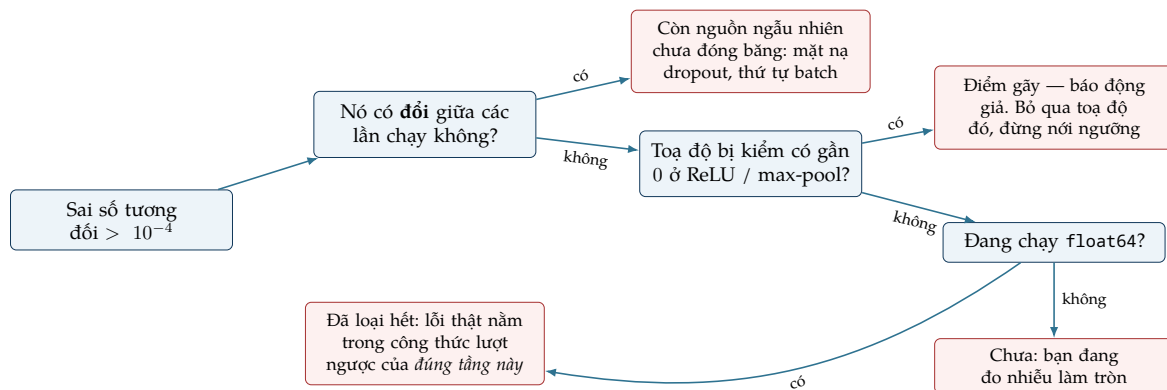
Ngưỡng diễn giải, theo *kinh nghiệm thực hành* chứ không phải kết quả lý thuyết:

- $< 10^{-7}$ — yên tâm.
- 10^{-7} đến 10^{-4} — chấp nhận được với mạng sâu có nhiều điểm gãy, đáng nghi với một tầng đơn giản.
- $> 10^{-4}$ — gần như chắc chắn có lỗi. $> 10^{-2}$ — chắc chắn có lỗi.

Ba chi tiết quyết định việc kiểm tra gradient nói thật hay nói dối:

1. **Phải dùng float64.** Sai phân trung tâm lấy hiệu hai số gần bằng nhau; với float32 (epsilon máy $\approx 1,2 \times 10^{-7}$) và $h = 10^{-5}$, bạn mất phần lớn chữ số có nghĩa vì mất chữ số có nghĩa khi trừ hai số gần bằng nhau — kết quả là bạn đang đo nhiễu làm tròn chứ không đo đạo hàm.
2. **Kiểm ở vài chục tọa độ ngẫu nhiên**, không phải toàn bộ tensor — vừa đủ tin cậy, vừa đủ nhanh để bạn thực sự chạy nó.
3. **Mọi nguồn ngẫu nhiên phải bị đóng băng giữa hai lần gọi f** ; nếu không, bạn đang so đạo hàm của hai hàm khác nhau.

Khi con số báo động, Hình 1.2 là thứ tự chẩn đoán tiết kiệm nhất.



Hình 1.2. Thứ tự chẩn đoán khi kiểm tra gradient báo động. Ba nhánh đầu là các nguyên nhân *giả*; chỉ khi loại hết chúng thì kết luận “công thức lượt ngược sai” mới có giá trị.

Kỷ luật đi kèm chỉ có một câu, và nó là phần đáng giá nhất của cả mục: *không ghép một tầng chưa qua kiểm tra*. Lý do thuần túy là số học tổ hợp. Ghép sáu tầng rồi mới phát hiện gradient sai thì không gian nghi vấn là sáu tầng cộng mọi tương tác giữa chúng; kiểm từng tầng ngay khi viết xong thì không gian nghi vấn luôn có đúng một phần tử. Đây chính xác là lập luận bạn đã dùng khi thêm từng loại ràng buộc vào một **đồ thị nhân tố** (*factor graph*) và kiểm tra sau mỗi lần thêm.

↪ **trường hợp riêng của A/01** — lan truyền ngược là chế độ nghịch của quy tắc chuỗi, nên kiểm tra gradient ở đây đúng là bước so Jacobian giải tích với Jacobian sai phân trong tối ưu phi tuyến.

E. Bước 4 — ghép thành mạng và tự viết vòng lặp

Bước cuối là ghép các tầng đã kiểm thành một mạng n tầng và tự viết vòng lặp huấn luyện với SGD kèm *momentum*. Ở đây bạn gặp một loại lỗi mới mà kiểm từng tầng không bắt được: lỗi về *trạng thái*.

- **Mặt nạ dropout** phải giống nhau giữa lượt xuôi và lượt ngược của cùng một bước, nhưng khác nhau giữa các bước.
- **BatchNorm** phải cập nhật thống kê chạy ở chế độ train và dùng thống kê ấy ở chế độ test.
- **Bộ đệm momentum** phải được cập phát một lần, không phải mỗi bước.

Ba lỗi này là ba phiên bản của cùng một câu hỏi: *cái gì là tham số, cái gì là trạng thái, cái gì là tạm*. Chính câu hỏi đó quay lại nguyên vẹn ở mục sau, dưới cái tên parameter so với buffer.

F. Tiêu chí dừng, giới hạn và trường hợp hỏng

Tiêu chí dừng nên phát biểu bằng năng lực giải thích, không bằng số bài đã làm. Bạn dừng được khi giải thích trôi chảy hai điều. Thứ nhất, vì sao BatchNorm khiến lượt ngược tốn thêm bộ nhớ: hai tổng trên toàn batch, nên không giải phóng được theo từng mẫu. Thứ hai, vì sao dropout phải scale lúc train chứ không lúc test: để bảo toàn kỳ vọng mà không đặt bước bắt buộc nào lên đường suy luận. Đạt rồi thì chuyển sang PyTorch và không viết NumPy nữa.

Giới hạn của cả giai đoạn cần nói thẳng:

- **Mọi trực giác về hiệu năng rút ra từ đây đều sai.** NumPy thuần chậm hơn nhiều bậc so với kernel thật, nên bạn chỉ chạy được ở quy mô đồ chơi; phần hiệu năng phải học lại ở B/11-gioi-han-tinh-toan.
- **Không học được gì về ổn định số học ở độ chính xác thấp.** *Log-sum-exp*, *loss scaling*, tràn số fp16 chỉ xuất hiện khi bạn rời float64.
- **Kiểm tra gradient kiểm tính nhất quán, không kiểm tính đúng.** Một tầng cài sai công thức lượt xuôi nhưng có lượt ngược khớp với chính nó sẽ qua kiểm một cách hoàn hảo.

CẠM BÃY

Bốn cách làm cho kiểm tra gradient qua trong khi mã vẫn sai, xếp theo mức độ hay gặp. **Một:** quên cố định mặt nạ dropout, khiến $f(x + h)$ và $f(x - h)$ là hai hàm khác nhau — triệu chứng là sai số lớn và *nhảy loạn giữa các lần chạy*. **Hai:** để BatchNorm ở chế độ train trong lúc kiểm, khiến hàm bị kiểm phụ thuộc thống kê batch và thay đổi khi bạn nhiễu một toạ độ. **Ba:** kiểm ở đúng điểm gãy của ReLU hoặc max-pool, tức x rất gần 0. Ở đó sai phân hai phía đo qua một chỗ hàm không khả vi, nên báo động là giả. Cách xử lý đúng là bỏ qua toạ độ đó, không phải nới ngưỡng. **Bốn:** chạy ở float32 rồi kết luận rằng “sai số 10^{-3} là bình thường”; không, đó là dấu hiệu bạn đang đo nhiễu làm tròn.

CẦN NHỚ

Không ghép một tầng chưa qua kiểm tra gradient. Lý do không phải sự cẩn thận mà là tổ hợp: kiểm từng tầng giữ không gian nghi vấn ở đúng một phần tử; ghép trước rồi kiểm sau làm nó phình lên thành số tầng nhân với số tương tác. Và nhớ rằng kiểm tra gradient chỉ chứng minh lượt xuôi và lượt ngược *khớp nhau*, không chứng minh lượt xuôi đúng. [T2]

G. Kết nối

Mục này là tiền đề bắt buộc của [D/19/02-pytorch-thanh-thao](#). Ba khái niệm bạn vừa cài bằng tay — đồ thị tính toán, thứ phải nhớ giữa hai lượt, và tham số so với trạng thái — sẽ xuất hiện lại dưới dạng autograd, nn.Module và cặp `train()/eval()`. Bạn sẽ hiểu chúng như tên gọi mới của thứ đã biết. Nó là trường hợp riêng của [A/01-toan-toi-thieu/02-vi-phan-va-toi-uu](#): backpropagation chỉ là chế độ nghịch của quy tắc chuỗi áp cho hàm hợp [4]. Nó bổ sung cho [C/12-co-che-huan-luyen](#): chương kia nói BatchNorm và dropout *làm gì* cho việc huấn luyện, mục này nói chúng *tồn gì* và hỏng ở đâu. Và nó gieo trước một hạt cho [D/22-tai-lap-va-mlops](#): phép cộng dồn tán xạ trong *col2im* là nguồn bất định GPU mà bạn sẽ gặp lại ở đó.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao hinge loss cho gradient bằng đúng không ở một mẫu đã thoả lề, còn softmax thì không bao giờ? Hệ quả nào của điều đó xuất hiện khi tập dữ liệu có nhãn sai?
2. Hai tổng trên toàn batch trong lượt ngược của BatchNorm là gì, và chúng buộc ta giữ thêm gì trong bộ nhớ?
3. Vì sao *inverted dropout* tốt hơn về mặt kỹ thuật dù hai cách scale tương đương về toán học? Và nó *không* bảo toàn thứ gì?
4. Kiểm tra gradient cho sai số tương đối 3×10^{-3} và con số đó đổi mỗi lần chạy. Nguyên nhân khả dĩ nhất là gì, và nếu con số ổn định thì danh sách nghi vấn đổi thế nào?
5. Kiểm tra gradient qua hết mọi tầng nhưng mạng vẫn không học — hai loại lỗi nào mà kiểm tra gradient theo thiết kế không thể bắt được?
6. Chỗ nào trong lượt ngược của convolution là phép cộng dồn tán xạ, và vì sao chi tiết đó sẽ quay lại ám bạn khi chạy trên GPU?

Nguồn

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.

1.2 Giai đoạn 2: PyTorch thành thạo (Becoming Fluent in PyTorch)

PHỤ THUỘC [D/19/01-tu-viet-bang-numpy](#) (bắt buộc — mọi khái niệm ở đây là tên gọi mới của thứ bạn đã cài bằng tay); [B/11-gioi-han-tinh-toan](#) (*roofline*, memory-bound so với compute-bound — không có nó thì mục về tăng tốc chỉ là danh sách mẹo).

TÓM TẮT

Mục này đi theo mạch vấn đề → công cụ: mỗi tính năng của PyTorch được giới thiệu qua đúng cái bug hoặc cái nút thắt đã sinh ra nó, chứ không theo thứ tự tài liệu API. Đọc xong bạn đọc được một *profile* và biết nên bật *AMP*, sửa *DataLoader*, hay gọi `torch.compile` — thay vì bật cả ba rồi không biết cái nào có tác dụng. Nếu chỉ nhớ một điều: mọi công cụ tăng tốc ở đây chỉ giúp khi bạn đang nghẽn ở đúng chỗ nó gỡ, nên thứ tự bắt buộc luôn là *đo trước, sửa sau*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tensor** — mảng nhiều chiều kèm kiểu dữ liệu (*dtype*) và thiết bị (*device*, CPU hay GPU). Trong bộ nhớ nó chỉ là một dải liên tục cộng với bộ *stride* nói mỗi chiều nhảy bao nhiêu phần tử.
- **đồ thị tính toán (computation graph)** — bản ghi các phép toán đã thực hiện ở lượt xuôi, để lượt ngược biết đi ngược theo đường nào. PyTorch dựng nó *động*, tức mỗi lần lượt xuôi là một đồ thị mới.
- **epoch, batch, DataLoader** — một *epoch* là một lượt đi hết dữ liệu; *batch* là nhóm mẫu xử lý cùng lúc; *DataLoader* là bộ máy đọc đĩa, giải mã, augment và gom mẫu thành batch — thường chạy trên CPU và thường là chỗ nghẽn.
- **độ chính xác hỗn hợp (mixed precision)** — tính phần lớn phép toán ở 16 bit trong khi giữ trọng số và một số phép cộng dồn ở 32 bit. Mục đích là giảm số byte phải nạp qua bộ nhớ, không phải giảm số phép tính.
- **memory-bound so với compute-bound** — một kernel memory-bound bị chặn bởi băng thông bộ nhớ (làm ít phép tính trên mỗi byte nạp về), compute-bound thì bị chặn bởi năng lực tính toán. Hầu hết mạng thị giác là memory-bound — lý do AMP có tác dụng.

- **profiler** — công cụ đo thời gian thực sự của từng kernel và từng giai đoạn, thay vì đoán. Trong PyTorch là `torch.profiler`; ở tầng dưới là `Nsight`.
- **checkpoint và state_dict** — `state_dict` là từ điển chứa toàn bộ trọng số và *buffer* của mô hình; lưu nó ra đĩa được gọi là checkpoint. Nó *không* chứa mã nguồn, nên vẫn cần đúng phiên bản code để nạp lại.
- **tính tất định (determinism)** — tính chất chạy lại cùng đầu vào cho ra cùng đầu ra tới từng bit. Trên GPU nó không mặc định, và ép nó thì phải trả giá bằng tốc độ.

A. Đặt vấn đề

Sau giai đoạn 1 bạn biết cơ chế nhưng chưa có công cụ. PyTorch cung cấp công cụ, nhưng nó cung cấp *rất nhiều* công cụ, và tài liệu chính thức tổ chức theo API chứ không theo vấn đề. Hệ quả là kiểu hiểu biết phổ biến nhất: biết gọi `amp.autocast` mà không biết nó giúp gì; bật `torch.compile` rồi thấy không nhanh hơn và kết luận rằng “compile không ăn thua”.

Mục này đảo lại thứ tự trình bày. Mỗi công cụ xuất hiện sau đúng cái vấn đề đã sinh ra nó, vì trong nghề, thứ bạn thực sự cần nhớ không phải tên hàm — tra được trong ba mươi giây — mà là *ánh xạ từ triệu chứng sang công cụ*. Cần nói rõ đây là nội dung tầng bốn [T4]: tên API sẽ đổi, và điều đó bình thường; cái không đổi là danh sách các nút thắt.

→ **tiền đề D/19/01** — `autograd`, `buffer` và cặp `train/eval` chỉ là tên gọi mới của đồ thị tính toán, trạng thái và công tắc chế độ mà bạn đã tự cài bằng tay.

Học xong thì làm được gì? Bạn cầm một vòng lặp huấn luyện chậm gấp ba lần đáng ra, chạy *profiler*, và chỉ đúng một dòng cần sửa — đây chính là kỹ năng bạn đã có với `nsys` và `perf` ở phía C++/CUDA, lần này áp cho một *runtime* có thêm một tầng trừu tượng ở giữa.

TRỰC GIÁC

Autograd là một cuốn băng ghi âm chỉ dùng được một lần. Mỗi phép toán trên *tensor* có `requires_grad=True` ghi thêm một mục vào băng; `backward()` tua ngược băng, tính gradient, rồi *xoá* băng đi. Từ hình ảnh đó suy ra ba điều mà người ta thường phải học bằng cách vấp: gọi `backward()` hai lần thì lỗi vì băng đã bị xoá (trừ khi bạn xin `retain_graph`); giữ một *tensor* còn dính băng trong một danh sách log nghĩa là giữ luôn toàn bộ đoạn băng phía sau nó, nên bộ nhớ phình dần suốt *epoch*; và `detach()` chỉ đơn giản là cắt băng ở đúng chỗ đó.

B. Mạch vấn đề → công cụ

Chín nút thắt, theo thứ tự bạn sẽ gặp

- 1 Vấn đề** bug im lặng về bộ nhớ và tính đúng đắn.

Giải pháp hiểu *tensor* ở mức bố cục bộ nhớ: `dtype`, `device`, *stride* (mỗi bước theo một chiều nhảy bao nhiêu phần tử trong bộ nhớ), và khác biệt *view* với *copy*.

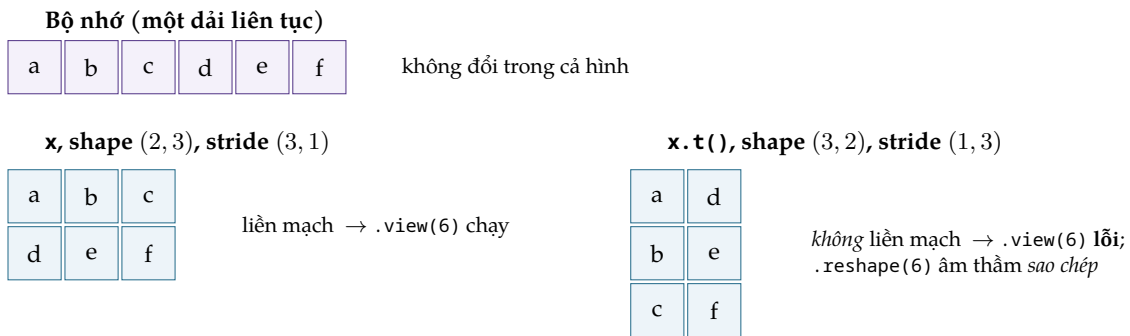
Hệ quả `.view()` thất bại trên *tensor* không liên mạch, còn `.reshape()` âm thầm sao chép — một `.contiguous()` đặt sai chỗ có thể nhân đôi bộ nhớ mà không báo gì.
- 2 Vấn đề** không biết đồ thị được dựng và giải phóng lúc nào.

Giải pháp *autograd*: `backward`, `no_grad`, `detach`, và `autograd.Function` tùy biến khi bạn có công thức gradient riêng.

- Vấn đề mới** bug kinh điển: giữ tensor còn đồ thị trong một list để logging, làm rò bộ nhớ suốt epoch.
- 3 Vấn đề** trạng thái mô hình không chỉ là trọng số.
Giải pháp nn.Module: phân biệt *parameter* với *buffer*, hiểu state_dict, dùng *hook*, và cặp train()/eval().
Giả định bạn nhớ rằng thống kê chạy của BatchNorm là *buffer* chứ không phải tham số — nó được lưu, được chuyển device, nhưng không được optimizer đụng tới.
- 4 Vấn đề** GPU đói dữ liệu.
Giải pháp Dataset/DataLoader/sampler, num_workers, pin_memory, *prefetch*, collate_fn; và một tầng *augmentation* (*alumentations*, transforms v2, hoặc DALI khi CPU không kịp).
Hệ quả đây là nút thắt phổ biến nhất và cũng bị bỏ qua nhiều nhất — xem B/11-gioi-han-tinh-toan.
- 5 Vấn đề** framework giấu quá nhiều.
Giải pháp tự viết vòng lặp huấn luyện ít nhất một lần trước khi dùng Lightning hay HF Trainer.
Giá phải trả vài giờ.
Hệ quả sau đó dùng framework là hợp lý, vì bạn đã biết nó đang làm gì và biết chỗ để chọc vào khi nó làm sai.
- 6 Vấn đề** bộ nhớ và tốc độ.
Giải pháp *AMP* (tính phần lớn phép toán ở 16 bit, giữ trọng số ở 32 bit) với GradScaler; *DDP* (chia batch cho nhiều GPU, mỗi GPU giữ một bản sao mô hình) qua torchrun, hoặc FSDP khi một GPU không chứa nổi mô hình [5]; và torch.compile để hợp nhất kernel và giảm chi phí điều phối.
- 7 Vấn đề** đo trước khi tối ưu.
Giải pháp torch.profiler, Nsight, trace viewer.
Giả định bạn tin số đo hơn tin trực giác — nguyên tắc đã có ở B/11-gioi-han-tinh-toan.
- 8 Vấn đề** kết quả không lặp lại.
Giải pháp *seed*, torch.use_deterministic_algorithms, tắt cudnn.benchmark — kèm cái giá về tốc độ.
Giá phải trả đáng kể, và *vẫn không* đảm bảo tái lập giữa hai máy khác GPU. Toàn bộ chủ đề này thuộc D/22-tai-lap-va-mlops.
Hệ quả biết là đủ: viết kernel riêng bằng CUDA hoặc Triton. Cần khi bạn có một mẫu truy cập bộ nhớ đặc thù mà không kernel nào khớp — hiếm, và thường có cách rẻ hơn.

C. Stride, view và copy — bố cục bộ nhớ là thứ thật

Đây là chỗ người từ C++ có lợi thế lớn nhất, vì *stride* chính là thứ bạn vẫn dùng khi viết kernel: một tensor là một con trỏ, một *shape*, và một bộ *stride* nói mỗi bước theo mỗi chiều nhảy bao nhiêu phần tử. Hình 1.3 cho thấy vì sao chuyển vị không tốn gì và vì sao .view() sau đó lại thất bại.



Hình 1.3. Chuyển vị chỉ đổi *stride*, không chạm vào bộ nhớ — nên nó gần như miễn phí. Nhưng `.view()` đòi hỏi các phần tử nằm đúng thứ tự trong bộ nhớ, nên nó thất bại; `.reshape()` thì không thất bại mà lặng lẽ cấp phát một bản sao. Chính sự “lặng lẽ” đó là nguồn của các lần bùng nổ bộ nhớ không giải thích được.

Quy tắc thực hành rút ra: dùng `.view()` khi bạn *muốn* chương trình báo lỗi nếu bố cục không như bạn nghĩ, và dùng `.reshape()` chỉ khi bạn đã chấp nhận cái giá sao chép. Đây là cùng một triết lý với `at()` so với `operator[]` trong C++: chọn phiên bản ồn ào ở chỗ bạn chưa chắc.

D. Parameter, buffer và cặp train/eval

Ở giai đoạn 1 bạn đã phải tự trả lời “cái gì là tham số, cái gì là trạng thái”. PyTorch đặt tên cho câu trả lời đó:

- **Parameter** — được optimizer cập nhật, có gradient, nằm trong `state_dict`. Ví dụ: γ, β của BatchNorm.
- **Buffer** — *không* có gradient và optimizer không đụng tới, nhưng vẫn nằm trong `state_dict` và vẫn theo mô hình khi bạn gọi `.to(device)`. Ví dụ: `running_mean`, `running_var` của BatchNorm.
- **Biến tạm** — không lưu, không chuyển; nếu bạn cần nó tồn tại qua một lần lưu checkpoint thì bạn đã chọn sai loại.

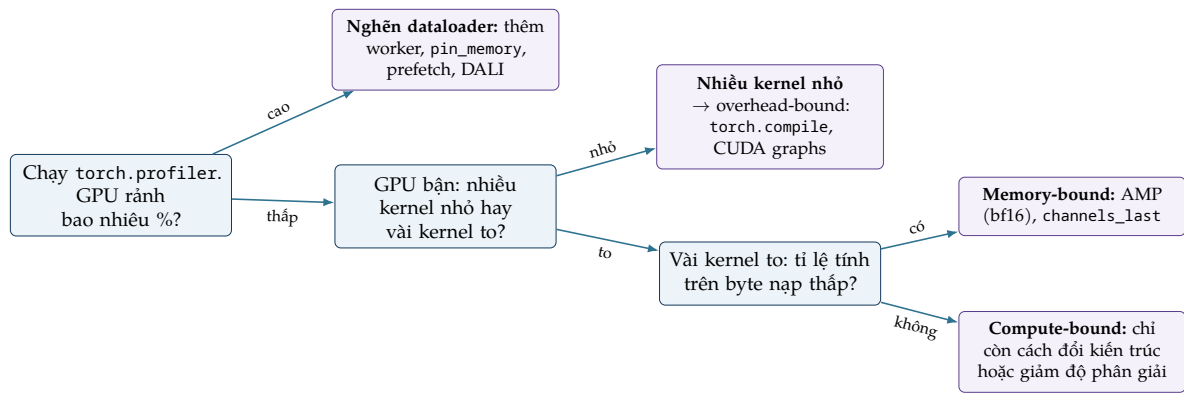
Từ đó suy ra vì sao quên `model.eval()` là lỗi kinh điển và im lặng: ở chế độ train, BatchNorm dùng thống kê của *batch hiện tại* thay vì buffer, và dropout bật lại. Kết quả là đầu ra phụ thuộc vào việc bạn tình cờ đưa mẫu nào vào cùng batch — một mẫu đánh giá có thể ra hai kết quả khác nhau chỉ vì bạn đổi thứ tự dữ liệu. Không có exception nào; chỉ có một con số hơi lệch.

↪ **hệ quả của C/12** — BatchNorm được đưa vào để ổn định huấn luyện; chính cơ chế ấy làm mô hình phụ thuộc cả batch, nên nó kéo theo hai chế độ hành vi khác nhau.

E. Tăng tốc: đo trước, và biết mỗi đòn bẫy gỡ nút thắt nào

Đây là phần bị lạm dụng nhiều nhất, nên bắt đầu bằng nguyên tắc: *mỗi đòn bẫy chỉ giúp khi bạn đang nghẽn ở đúng chỗ nó gỡ*. Hình 1.4 là cây quyết định rút gọn, và Bảng 1.2 là phần chi tiết.

↪ **ràng buộc bởi B/11** — cây quyết định tăng tốc chỉ là mô hình roofline diễn đạt bằng tên API: nhánh nào đúng phụ thuộc bạn đang bị chặn bởi băng thông, năng lực tính, hay chi phí điều phối.



Hình 1.4. Cây quyết định tăng tốc một vòng lặp huấn luyện. Nhánh đầu tiên — GPU đang rảnh — là nhánh đúng trong phần lớn trường hợp thực tế, và cũng là nhánh mà mọi mẹo về `torch.compile` đều vô dụng.

Bảng 1.2. Năm đòn bẩy tăng tốc, và nút thắt mà mỗi cái gỡ.

Đòn bẩy	Cơ chế	Gỡ nút thắt nào	Hỏng / vô dụng khi nào
Sửa DataLoader	Nhiều <i>worker</i> , <i>pin_memory</i> , <i>prefetch</i> , chuyển augmentation lên GPU	GPU đói dữ liệu	Quá nhiều worker làm cạn RAM và tăng thời gian khởi động; augmentation trên GPU tranh chấp với training
AMP (fp16/bf16)	Nửa số byte phải nạp và ghi	Memory-bound	Compute-bound thuần; fp16 tràn số khi không có GradScaler
<code>channels_last</code>	Đổi bố cục sang NHWC cho <i>Tensor Core</i>	Conv memory-bound	Mô hình có op không hỗ trợ NHWC → thêm phép hoán vị, chậm hơn
<code>torch.compile</code>	Hợp nhất kernel, giảm chi phí điều phối Python	Overhead-bound	Shape động gây biên dịch lại liên tục; <i>graph break</i> do code Python động
DDP / FSDP	Chia batch (DDP) hoặc chia trọng số (FSDP) qua nhiều GPU	Không đủ thời gian hoặc bộ nhớ	Bảng thông liên kết kém → <i>all-reduce</i> nuốt hết lợi ích; batch hiệu dụng đổi nên phải chỉnh <i>learning rate</i> [6]

Ví dụ nhỏ nhất cho AMP. Một tensor kích hoạt cỡ $64 \times 256 \times 56 \times 56$ có 51,4 triệu phần tử. Ở fp32 nó chiếm 205 MB; ở bf16 chiếm 103 MB. Trên một GPU có băng thông 900 GB/s, chỉ riêng việc đọc nó một lần tốn $228 \mu\text{s}$ so với $114 \mu\text{s}$. Ý nghĩa: nếu tăng đó chỉ làm vài phép tính trên mỗi byte nạp về, AMP cắt gần một nửa thời gian; nếu nó làm rất nhiều phép tính trên mỗi byte, AMP gần như không đổi gì. Đó chính là trục *arithmetic intensity* của B/11-gioi-han-tinh-toan, xuất hiện lại dưới dạng một lựa chọn hai dòng code.

fp16 hay bf16? Hai lựa chọn đối lập rõ ràng nên đáng đặt cạnh nhau: fp16 có 10 bit phần định trị và dải mũ hẹp, nên chính xác hơn ở vùng giá trị bình thường nhưng tràn hoặc triệt tiêu về không khi gradient nhỏ — vì thế nó *bắt buộc* cần GradScaler để nhân loss lên trước khi lượt ngược [7]. bf16 có cùng dải mũ với fp32 nhưng chỉ 7 bit định trị: kém chính xác hơn, đổi lại gần như không bao giờ tràn và không cần GradScaler. Trên phần cứng hỗ trợ bf16, mặc định hợp lý là bf16; fp16 chỉ còn lý do tồn tại trên phần cứng cũ.

```

scaler = torch.amp.GradScaler() # chỉ cần cho fp16
model = model.to(memory_format=torch.channels_last)
for x, y in loader:
    x = x.cuda(non_blocking=True).to(memory_format=torch.channels_last)
    opt.zero_grad(set_to_none=True) # set_to_none re hơn zero_
    with torch.amp.autocast("cuda", dtype=torch.bfloat16):
        loss = criterion(model(x), y.cuda(non_blocking=True))
    scaler.scale(loss).backward()
    scaler.step(opt); scaler.update()
  
```

F. Giới hạn và trường hợp hỏng

- **torch.compile không phải nút bấm thần kỳ.** Nó giúp khi bạn đang overhead-bound; nó không giúp gì cho mô hình nghẽn ở dataloader, và nó có thể *chậm hơn* nếu shape đầu vào thay đổi liên tục khiến nó biên dịch lại. Với dữ liệu ảnh có kích thước thay đổi hoặc số hộp phát hiện thay đổi, đây là tình huống thường gặp chứ không phải ngoại lệ.
- **Profiler cũng nói dối nếu bạn đo sai.** Lệnh CUDA là bất đồng bộ, nên đo bằng đồng hồ Python quanh một lời gọi chỉ đo thời gian *xếp hàng*, không đo thời gian chạy. Phải đồng bộ hoá hoặc dùng CUDA event — một cái bẫy bạn đã gặp ở phía C++, và nó không biến mất khi lên Python.
- **DDP đổi ngữ nghĩa của batch.** Batch hiệu dụng là batch mỗi GPU nhân số GPU, nên mọi siêu tham số phụ thuộc batch (learning rate, lịch warm-up, thống kê BatchNorm) đều đổi theo. Một so sánh “1 GPU với 8 GPU” mà không chỉnh learning rate là một so sánh vô nghĩa.
- **Tắt định có giá.** Bật use_deterministic_algorithms buộc PyTorch chọn kernel chậm hơn hoặc báo lỗi ở những op không có bản tắt định — và nó vẫn không đảm bảo hai máy khác GPU cho cùng kết quả.

CẠM BẤY

Rò bộ nhớ vì logging là bug tốn thời gian nhất trong danh sách này, vì triệu chứng xuất hiện sau vài trăm bước chứ không phải ngay lập tức. Viết `losses.append(loss)` thì bạn đang giữ luôn toàn bộ đồ thị tính toán dẫn tới loss, kể cả mọi activation trung gian; sau một epoch, GPU hết bộ nhớ và *stack trace* chỉ vào một dòng hoàn toàn vô can. Cách đúng là `losses.append(loss.detach())` hoặc `loss.item()`. Cùng một họ lỗi: giữ output của mô hình trong một *dict* để debug, hoặc trả tensor còn đồ thị ra khỏi một hàm được gọi trong vòng lặp.

CẦN NHỚ

Ánh xạ triệu chứng → công cụ mới là thứ đáng nhớ, không phải tên API. GPU rảnh thì sửa dataloader; nhiều kernel nhỏ thì `torch.compile`; ít tính trên mỗi byte thì AMP và `channels_last`; không đủ bộ nhớ thì FSDP hoặc *gradient checkpointing*. Và trước tất cả: chạy profiler, vì tối ưu thứ chưa đo là cách chắc chắn nhất để mất một tuần mà không nhanh hơn. [T2]

G. Kết nối

Mục này là bản dịch của [D/19/01-tu-viet-bang-numpy](#) sang ngôn ngữ công cụ: *autograd* là cuốn băng bạn đã tự cuộn bằng tay, *buffer* là “trạng thái” bạn đã tự quản lý, `eval()` là cái công tắc bạn đã tự viết. Nó là ứng dụng trực tiếp của B/11-gioi-han-tinh-toan: cây quyết định ở Hình 1.4 chỉ là mô hình *roofline* được diễn đạt bằng tên API. Nó là tiền đề của D/20-toi-uu-va-nen-mo-hinh (nơi cùng những đòn bẫy này được áp cho **suy luận** (*inference*) thay vì huấn luyện) và của D/22-tai-lap-va-mlps (nơi mục về *seed* và tính tắt định được mở rộng thành một chương).

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Khi nào `.view()` thất bại mà `.reshape()` thì không, và điều đó liên quan gì tới *stride*? Vì sao thất bại đôi khi lại là hành vi bạn muốn?
2. Vì sao `torch.compile` không giúp gì cho mô hình bị nghẽn ở dataloader, và làm sao biết mình đang ở tình huống đó trong ba mươi giây?
3. Thống kê chạy của BatchNorm là *parameter* hay *buffer*, và câu trả lời đó giải thích thế nào cho việc quên `eval()` lại làm kết quả phụ thuộc thứ tự dữ liệu?

4. Vì sao fp16 cần GradScaler mà bf16 thì không? Cái giá của bf16 là gì?
5. Bạn đo một lời gọi mô hình bằng `time.time()` và thấy nó tốn 0,2 ms. Vì sao con số đó gần như chắc chắn sai, và phải đo thế nào?
6. Chuyển từ 1 GPU sang 8 GPU bằng DDP, mọi thứ chạy nhưng độ chính xác tụt — hai nguyên nhân khả dĩ nhất là gì, biết rằng cả hai đều không phải bug trong DDP?

Nguồn

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.

1.3 Thực hành

Hai mini project dưới đây được thiết kế để *đóng* giai đoạn 1 và *mở* giai đoạn 2. Cái thứ nhất chứng minh bạn đã hiểu đúng cơ chế; cái thứ hai chứng minh bạn tăng tốc được dựa trên số đo.

Repo và tài nguyên. Để học: bài tập CS231n (làm hết assignment 1 và 2), [karpathy/micrograd](#) (autograd trong hai trăm dòng), [karpathy/nn-zero-to-hero](#), [pytorch/examples](#). Để dùng thật: [huggingface/pytorch-image-models](#) (timm), [Lightning-AI/pytorch-lightning](#), [facebookresearch/hydra](#), [NVIDIA/DALI](#), [alumentations](#). Để hiểu framework từ bên trong: [tinygrad/tinygrad](#) — đủ nhỏ để đọc hết. Danh sách này chốt tại tháng 9/2026; các repo hạ tầng ở đây (timm, DALI, Hydra) bền hơn nhiều so với tên mô hình dẫn đầu bảng, nhưng API của chính PyTorch cũng đổi, nên hãy nhớ vấn đề và tra lại cú pháp.

Câu hỏi kiểm tra hiểu. Khi nào `.view()` thất bại mà `.reshape()` thì không, và điều đó liên quan gì tới *stride*? Vì sao `torch.compile` không giúp gì cho mô hình bị nghẽn ở dataloader? *Hook* nào cần dùng để lấy activation ở giữa mạng mà không sửa `forward`? Và câu hỏi đóng giai đoạn 1: vì sao BatchNorm khiến lượt ngược tốn thêm bộ nhớ, còn dropout phải scale lúc train chứ không lúc test?

Mini project (vài giờ đến hai ngày). Viết lại *chính xác* một mạng NumPy của bạn bằng PyTorch và kiểm tra hai bên khớp tới 10^{-6} ở cả lượt xuôi lẫn gradient. Phần đáng giá không phải lúc nó khớp mà lúc nó không khớp: với mỗi chỗ lệch, phân loại xem đó là *bug* (lệch có hệ thống, không giảm khi bạn tăng độ chính xác) hay *số học* (lệch cỡ epsilon máy, đôi khi bạn đổi thứ tự phép cộng). Kỹ năng phân loại này là thứ bạn sẽ dùng lại mỗi lần export mô hình ở chương 21.

■ **Active recall** (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Lướt xuôi, lướt ngược	Lướt xuôi tính từ đầu vào ra loss; lướt ngược đi ngược để lấy đạo hàm theo từng tham số. Mọi vòng lặp huấn luyện là hai lượt này lặp lại.
●●● Lan truyền ngược	Thuật toán vi phân cho mọi đồ thị khả vi, tồn cùng bậc chi phí với một lướt xuôi. Đừng nhầm nó (thuật toán) với lướt ngược (một lần chạy).
●● Hinge so softmax	Hinge cho gradient bằng đúng không khi lề đã thỏa, nên ổn định. Softmax không bao giờ tắt, nên nó tinh chỉnh tiếp nhưng nhạy với nhiễu sai.
●●● Gradient check	So gradient giải tích với sai phân trung tâm ở vài chục tọa độ ngẫu nhiên. Nó chỉ chứng minh xuôi và ngược khớp nhau, không chứng minh lướt xuôi đúng.
●●● Bộ nhớ kích hoạt	Chỗ chứa đầu ra trung gian mà lướt ngược phải đọc lại. Đây là lý do huấn luyện tốn bộ nhớ hơn suy luận nhiều lần, và là thứ chặn batch size.
●● Vì sao lướt ngược BatchNorm không theo từng phần tử?	Nó chứa hai tổng trên toàn batch, một cho trung bình và một cho phương sai. Gradient của một mẫu vì thế phụ thuộc mọi mẫu khác trong batch.
●● Inverted dropout	Giữ phần tử với xác suất $1 - p$ rồi chia cho $1 - p$ lúc train; lúc test không làm gì. Bảo toàn kỳ vọng mà không đặt bước bắt buộc nào lên đường suy luận.
●● Vì sao col2im sinh ra bất định trên GPU?	Nó cộng dồn gradient vào các vị trí cửa sổ chồng lấn, và cài đặt GPU dùng atomicAdd. Cộng số thực không kết hợp, nên thứ tự luồng quyết định bit cuối.
●● Vì sao gradient check phải chạy float64?	Sai phân trung tâm lấy hiệu hai số gần bằng nhau. Ở float32, epsilon máy khoảng $1,2 \times 10^{-7}$ nên bạn đo nhiễu làm tròn chứ không đo đạo hàm.
●●● Autograd	Cuốn băng ghi lại các phép toán lúc lướt xuôi, tua ngược khi gọi backward() rồi xoá. Giữ một tensor còn dính băng nghĩa là giữ luôn cả đoạn băng phía sau nó.
●● Stride	Mỗi bước theo một chiều nhảy bao nhiêu phần tử trong bộ nhớ. Chuyển vị chỉ đổi stride nên miễn phí, nhưng làm tensor không còn liền mạch.
●● view so với reshape	view đòi tensor liền mạch và báo lỗi nếu không; reshape lạng lẽ sao chép. Dùng view ở chỗ bạn muốn chương trình ồn ào.
●●● Parameter so với buffer	Parameter được optimizer cập nhật; buffer thì không, nhưng vẫn nằm trong state_dict và vẫn đổi thiết bị theo mô hình. Thống kê chạy của BatchNorm là buffer.
●● Vì sao quên eval() làm kết quả đổi theo batch?	Ở chế độ train, BatchNorm dùng thống kê của batch hiện tại thay vì buffer, và dropout bật lại. Một mẫu vì thế cho hai kết quả khác nhau tùy bạn xếp nó cùng ai.
●●● AMP	Tính phần lớn phép toán ở 16 bit, giữ trọng số ở 32 bit. Nó giảm số byte phải nạp chứ không giảm số phép tính, nên vô ích với kernel compute-bound.
●● torch.compile	Hợp nhất kernel và cắt chi phí điều phối Python. Chỉ giúp khi bạn overhead-bound; shape thay đổi liên tục làm nó biên dịch lại và chậm hơn.
●● Sai số gradient check khoảng 3×10^{-3} và đổi mỗi lần chạy?	Còn một nguồn ngẫu nhiên chưa đóng băng: mặt nạ dropout, hoặc BatchNorm đang ở chế độ train. Con số ổn định thì mới nên nghi công thức.
●● Bật AMP và torch.compile mà epoch không nhanh hơn?	Bạn đang ngهن ở dataloader, không phải ở GPU. Chạy profiler xem GPU rảnh bao nhiêu phần trăm trước khi độn vào mô hình.
●● Bộ nhớ GPU phình dần suốt epoch rồi hết?	Bạn đang giữ tensor còn đồ thị trong một danh sách log. Dùng loss.detach() hoặc loss.item().

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
PyTorch, JAX	Toàn bộ autograd là hiện thực của vi phân chế độ nghịch trên đồ thị động — đúng thứ bạn tự cuộn bằng tay ở giai đoạn 1	Hạ tầng, ổn định từ 2017 tới nay
FlashAttention	Đánh đổi tính lại lượt xuôi để giảm bộ nhớ kích hoạt, tức chính <i>gradient checkpointing</i> áp cho attention	Đã thành mặc định trong gần như mọi thư viện transformer
timm	Vòng lặp huấn luyện chuẩn với AMP, channels_last và torch.compile — đúng bốn đòn bẩy của Bảng 1.2	Hạ tầng thị giác; công thức trong repo đáng đọc hơn tài liệu
FSDP và ZeRO	Chia trọng số và trạng thái optimizer thay vì nhân bản chúng, khi một GPU không chứa nổi mô hình	Ý tưởng từ 2020, vẫn là nền của mọi huấn luyện quy mô lớn
tinygrad, micrograd	Chính bài tập giai đoạn 1, viết đủ nhỏ để đọc hết trong một buổi	Giá trị sư phạm, không phải công cụ sản xuất

MINI PROJECT SLAM · Tự cài lại tầng mô tả của frontend bằng NumPy · 1–2 ngày

Mục tiêu	làm đúng một lần cái việc mà framework luôn làm hộ, trên đúng khối bạn đang dùng thật: viết cả lượt xuôi lẫn lượt ngược cho một mạng mô tả đặc trưng, và không tin nó cho tới khi kiểm tra gradient số cho phép.
Dữ liệu	chính tập mảnh ảnh 32×32 dựng ở mini project chương 18, cùng nhãn cặp lấy từ liên kết dữ liệu của bản đồ.
Các bước	viết lượt xuôi cho một khối gồm <i>convolution</i> 3×3 , BatchNorm, ReLU, max-pool và một tầng <i>fully-connected</i> rồi chuẩn hoá L2 ra vector mô tả 32 chiều. Viết lượt ngược giải tích cho từng tầng. Chạy kiểm tra gradient ở float64 trên 20 toạ độ ngẫu nhiên của <i>mỗi</i> tham số, với mọi nguồn ngẫu nhiên đóng băng và BatchNorm đặt ở chế độ cố định. Chỉ khi mọi tầng qua ngưỡng mới ghép lại và huấn luyện bằng một hàm mất mát tương phản trên các cặp có nhãn.
Xong khi	sai số tương đối của kiểm tra gradient dưới 10^{-6} cho mọi tham số của mọi tầng. Toạ độ nằm sát điểm gãy của ReLU hoặc max-pool được phép bỏ qua, nhưng phải ghi rõ đã bỏ bao nhiêu và vì sao. Sau đó bản NumPy khớp bản PyTorch tới 10^{-6} ở cả lượt xuôi lẫn <i>gradient</i> .
Nối với	D/19 cho bảng “backward cần nhớ gì” và cây chẩn đoán; A/01 cho quy tắc chuỗi. Mô hình mô tả sinh ra ở đây chính là thứ chương 20 đem đi lượng tử hoá.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì vi phân chế độ nghịch trả về đạo hàm theo *mọi* tham số trong một lượt, còn vi phân thuận chỉ trả về theo một hướng mỗi lượt. Chi phí một lượt ngược cùng bậc với một lượt xuôi, nên tổng chi phí không phụ thuộc số tham số — đó là toàn bộ khoản tiết kiệm.
- A2. Vì backward phải đọc lại trạng thái của lượt xuôi: đầu vào cho tầng *fully-connected*, mặt nạ cho ReLU và dropout, $\hat{x}$ cùng hai tổng batch cho BatchNorm. Suy luận không cần lượt ngược nên giải phóng được ngay sau mỗi tầng; huấn luyện thì không.
- A3. Nó chỉ kiểm tính *nhất quán* giữa lượt xuôi và lượt ngược, không kiểm tính đúng của lượt xuôi. Một tầng cài sai công thức xuôi nhưng có ngược khớp với chính nó sẽ qua hoàn hảo; ngoài ra nó không bắt được lỗi trạng thái, lỗi nhân và lỗi ở các chỗ nối.
- A4. Vì mỗi đòn bẩy chỉ gỡ đúng nút thắt của nó. Nếu GPU đang đói dữ liệu thì thời gian nằm ở dataloader, và cả AMP lẫn torch.compile đều không chạm tới chỗ đó. Cây quyết định ở Hình 1.4 bắt đầu bằng “GPU rảnh bao nhiêu phần trăm” chính vì lý do này.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng một dây chuyền làm bánh gồm mười khâu, mỗi khâu có vài cái nút chỉnh. Bánh ra lò bị mặn. Câu hỏi là: nút nào ở khâu nào đã làm nó mặn, và mặn thêm bao nhiêu?

Cách ngây thơ là vặn thử từng nút rồi chạy lại cả mẻ bánh. Có một nghìn cái nút thì phải chạy một nghìn mẻ. Không ai làm nổi.

Mẹo là đi ngược dây chuyền đúng một lần. Đứng ở cuối, bạn biết bánh mặn hơn mong muốn bao nhiêu. Lùi về khâu thứ mười và hỏi: nếu khâu này đưa ra thứ mặn hơn một chút, bánh cuối mặn thêm bao nhiêu? Có câu trả lời rồi thì lùi tiếp về khâu chín, và cứ thế. Một lượt đi ngược chia được trách nhiệm cho cả nghìn cái nút, tốn xấp xỉ bằng một lượt làm bánh bình thường.

Cái giá rất cụ thể: muốn hỏi ngược thì mỗi khâu phải giữ lại mẫu bán thành phẩm của lượt xuôi. Xưởng vì thế cần kho chứa gấp nhiều lần, và đó đúng là lý do máy tính hết bộ nhớ khi huấn luyện chứ không hết khi chạy thật. Cái giá thứ hai: công thức chia trách nhiệm ở mỗi khâu do bạn tự viết, và nếu sai một dấu thì cả dây chuyền vẫn chạy êm, bánh vẫn ra, chỉ là không bao giờ ngon lên. Vì vậy phải thử tay từng khâu trước khi ghép.

Một câu: *Chạy xuôi một lần và giữ lại dấu vết, đi ngược một lần để chia trách nhiệm — rẻ hơn thử từng nút hàng nghìn lần, và cái giá đúng bằng chỗ chứa dấu vết đó.*

Tổng kết chương

Chương này đi từ chỗ tự tay cuộn một cuốn băng autograd trong NumPy tới chỗ đọc được một trace của `torch.profiler` và biết chính xác dòng nào cần sửa. Sợi chỉ xuyên suốt là *trạng thái*: cái gì được nhớ, cái gì được lưu, cái gì đổi hành vi giữa hai chế độ — và phần lớn bug im lặng trong deep learning là bug ở đúng ba câu hỏi đó.

Điều còn bỏ ngỏ là toàn bộ phía **suy luận** (*inference*). Ở đây ta chỉ quan tâm huấn luyện: bộ nhớ hoạt động, thông lượng, tính đúng đắn của gradient. Nhưng ràng buộc thật của một hệ thống trên thiết bị nằm ở phía kia: độ trễ đuôi, năng lượng, và tập toán tử mà runtime hỗ trợ. Cùng những đòn bẫy trong Bảng 1.2 sẽ xuất hiện lại ở D/20-toi-uu-va-nen-mo-hinh, nhưng với một hàm mục tiêu khác hẳn. Điều thứ hai còn bỏ ngỏ là tính tái lập: mục 19.2 chỉ nêu tên `use_deterministic_algorithms` và nói nó không đủ; vì sao nó không đủ, và mục tiêu đúng là gì, thuộc về D/22-tai-lap-va-mlops.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Chương chia hai giai đoạn theo tầng bán rã [T2][T3] và [T4]. Hãy nêu một hiểu biết từ giai đoạn 1 mà giai đoạn 2 *không* dạy được, và một sai lầm ở giai đoạn 2 mà giai đoạn 1 không giúp bạn tránh?
2. Cùng một triệu chứng “mạng không học” có thể do đạo hàm sai, do lẫn lộn trạng thái train và eval, hoặc do dữ liệu. Hai giai đoạn cho bạn những công cụ phát hiện khác nhau nào, và thứ tự dùng chúng nên là gì?
3. BatchNorm xuất hiện ở cả hai mục: một lần là hai tổng trên batch trong lượt ngược, một lần là buffer chứ không phải parameter. Hai góc nhìn đó nói cùng điều gì về BatchNorm, và điều đó giải thích thế nào cho hành vi khi lưu rồi nạp lại checkpoint?
4. Phép cộng dồn tán xạ trong lượt ngược của convolution ở giai đoạn 1 quay lại ám bạn ở giai đoạn 2 dưới dạng nào, và nó liên quan gì tới việc hai lần chạy cho kết quả không trùng nhau?
5. Vì sao “huấn luyện tốn bộ nhớ gấp nhiều lần suy luận” là kết luận rút ra được từ giai đoạn 1 nhưng chỉ đo được ở giai đoạn 2, và bạn đo thế nào để tách phần kích hoạt khỏi phần trạng thái optimizer?
6. Bạn bật `torch.compile` và AMP mà thời gian một epoch không đổi. Trước khi kết luận công cụ vô dụng, bạn đo ba đại lượng nào, và mỗi đại lượng loại bỏ giả thuyết nào?
7. Một mẹo tối ưu nghe được trên diễn đàn có tuổi thọ ngắn hơn hẳn hiểu biết ở giai đoạn 1. Bạn dùng tiêu chí nào để quyết định một mẹo có đáng đưa vào code nền của mình hay không?

Nguồn

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [5] S. Rajbhandari, J. Rasley, O. Ruwase, et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *SC*, 2020.
- [6] P. Goyal, P. Dollár, R. Girshick, et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.
- [7] P. Micikevicius, S. Narang, et al., “Mixed Precision Training,” in *ICLR*, 2018.